

NEXUS

Financial Resilience Intelligence

Microfinance stress detection, network exposure, propagation analysis, simulation, and targeted intervention

M# PROBLEM STATEMENT / When One Borrower Falls Behind - SDG 8: Decent Work and Economic Growth. NEXUS models when financial stress is isolated, independently deteriorating, or potentially exposed through connected borrowers, without automatically penalizing healthy members.

Version: 3.2 - Final Frozen Implementation Specification

Primary audience: M# judges, implementation agents, student developers, MFI/NBFC-MFI product reviewers

Implementation principle: Build the analytical core first; visual richness is layered on top of deterministic, testable, deployment-ready foundations.

CORE WORKFLOW

OBSERVE → DIAGNOSE → CONNECT → SIMULATE → INTERVENE → MEASURE

Document Map

Section	Purpose
1. Product Vision	What NEXUS is, who uses it, and the product thesis
2. Domain Model	Indian microfinance entities, liability structures, loans, repayment, income, relationships
3. Synthetic Data Design	How the simulated world works and why the generator is independent of the detector
4. Analytics & Targets	Stress definitions, target construction, model ladder, propagation labels
5. Leakage & Causality Governance	Hard rules preventing look-ahead, target leakage, and circularity
6. Propagation Engine	Transparent exposure/ripple mathematics and calibration
7. Simulation & Intervention	Deterministic scenarios, counterfactuals, assumptions, sensitivity
8. Product Experience	Six product areas and operational vs showcase UX
9. Explainability, Fairness & Responsible Data	Evidence stories, healthy-borrower protection, inclusivity, privacy
10. Evaluation	Metrics, statistical power, confidence intervals, scenario robustness
11. Architecture & API	Services, module boundaries, endpoints, implementation order
12. Testing & Definition of Done	Invariants, acceptance criteria, reproducibility
13. M# Traceability & Monetization	Direct mapping to judging criteria and business model
14. Agent Handoff	Exact implementation contract for Claude Code and Antigravity/Gemini
15. Review Checklist	Questions a skeptical technical judge should be able to answer
16. Final Scope Cut Line	Core vs stretch features if implementation time becomes constrained

1. Product Vision

PRODUCT THESIS

NEXUS is a decision-support system for microfinance institutions that detects emerging financial stress, separates individual deterioration from network exposure, models where stress may ripple through connected group structures, and compares targeted interventions while minimizing unnecessary action against healthy borrowers.

1.1 Problem interpretation

- The central analytical challenge is not merely detecting repayment problems. It is distinguishing: (a) isolated difficulty, (b) independent borrower deterioration, and (c) vulnerability associated with another borrower or shared structure.
- NEXUS therefore separates observed financial state, modeled network exposure, modeled propagation vulnerability, and simulated outcomes. These concepts must never be collapsed into one opaque score.
- Correlation is not treated as propagation: a seasonal or local economic downturn can stress several borrowers simultaneously without meaningful borrower-to-borrower transmission.

1.2 Primary users and jobs-to-be-done

User	Need	NEXUS output
Field / branch officer	Identify which borrowers need attention without alarming healthy members	Borrower queue, evidence trail, local-language explanation, low-bandwidth view
Risk / portfolio manager	Understand group and portfolio concentration	Group health, network hotspots, propagation paths, scenario analysis
Credit / collections lead	Compare targeted responses before acting	Intervention Studio with assumptions and sensitivity
Model / analytics reviewer	Verify whether graph/propagation adds predictive value	M0/A/B/C ablation, leakage tests, scenario metrics, confidence intervals
MFI product / technology lead	Integrate intelligence with existing LOS/LMS systems	API contract, role-based views, batch scoring, audit trail

2. Domain & Data Model

REALISM PRINCIPLE / The synthetic world follows Branch -> Centre -> Joint Liability Group (JLG) -> Borrower. JLG/group liability is the primary network channel; pairwise guarantees and informal dependencies are secondary. Repeat borrowing, household affordability, external debt, income seasonality, and shared economic exposure are first-class concepts.

2.1 Core entities

Entity	Key fields	Notes
Borrower	borrower_id, household_id, JLG/centre/branch ids, region, occupation, income sources, dependents	A person may have multiple loan cycles over time.
Household	household_id, income, expenses, external debt obligations, dependents	Supports affordability and multi-lender indebtedness context.
Branch	branch_id, region, centre ids	Operational hierarchy.
Centre	centre_id, branch_id, JLG ids,	Secondary social / coordination

Entity	Key fields	Notes
	region	layer.
JLG / Group	group_id, centre_id, members, liability rule, capacity	Primary joint-liability structure and network channel; default JLG size around 5.
Loan	loan_id, borrower_id, group_id, origination, principal, term, instalment plan, status	Supports multiple cycles and changing principal / instalment profiles.
Repayment Event	loan_id, due_date, amount due / paid, days past due, group coverage amount	Observed timestamped outcome; leakage-sensitive.
Income Record	borrower / household, date, source, inflow, regularity	Supports seasonality, concentration, and shock mechanics.
Expense / Cash-flow Record	date, essential, discretionary, debt service, buffer	Supports affordability and liquidity state.
External Debt	household_id, lender type, obligation, active dates	Captures multi-lender burden without pretending to be bureau data.
Relationship	member/member, member/group, shared source, guarantee, economic exposure	Every edge has semantic type, strength, start/end validity, and evidence.
Economic Context	region, period, sector / season indicators	Can create correlated pressure without borrower-to-borrower transmission.
Scenario Metadata	scenario_id, family, seed, split, shock parameters	Generation/evaluation metadata only; never predictive.
Hidden Lineage	source, mechanism, affected entity, lag, attribution share	Evaluation-only ground truth; never exposed to the model.

2.2 Network representation

Node / edge	Purpose	Priority
Borrower <-> JLG	Joint liability / membership	Primary
JLG <-> Centre	Secondary social / coordination exposure	Secondary
Borrower <-> Borrower	Guarantee, support, shared finances	Secondary
Borrower <-> Income Source	Concentration and common-source exposure	Primary
Borrower <-> Region / Sector	Common economic pressure	Secondary
Household <-> External Debt	Affordability / indebtedness	Primary for individual risk

Design rule: Do not model every observed relationship as a transmissible edge. JLG liability is aggregated as one group-level exposure channel; each relationship type must have an explicit semantic meaning, strength, validity period, and feature policy.

3. Synthetic Data & Scenario Design

CRITICAL SCIENTIFIC RULE

The synthetic data generator must use a different mechanism from the NEXUS propagation detector. The generator creates the simulated world; NEXUS tries to infer emerging risk from information available before the outcome. This prevents self-fulfilling evaluation.

3.1 Timeline and granularity

Dimension	Specification
Simulation horizon	At least 36 months so training, validation, and held-out test contain meaningful repeated patterns.
Financial state	Weekly observations for cash-flow / income / expenses where practical; monthly borrower-level aggregation for some dashboard summaries.
Repayments	Weekly or fortnightly cadence; exact due / payment timestamps retained.
Borrower count	Target 300-600 borrowers for primary evaluation; 200 acceptable for smoke tests.
Groups	Around 5 borrowers per JLG; centres contain multiple JLGs, roughly 15-30 borrowers. Configurable.
Loan cycles	Multiple loan cycles per borrower with realistic origination gaps and changing principal / instalment profiles.
Split	Chronological train < validation < held-out test; no scenario episode may straddle a split; add purge gaps.
Seeds	10 or more world-generation seeds for primary evaluation; 1-2 for rapid development tests.

3.2 Scenario families

Scenario	Underlying mechanism	What it tests
seasonal_downturn	Regional/sector income pressure affects several borrowers with recovery. Some group coverage may occur; borrower-level propagation labels depend on	Hard negative: correlated stress without contagion

Scenario	Underlying mechanism	What it tests
	hidden lineage. Hard negative for correlation vs propagation.	
isolated_shock	One borrower has a personal income/expense shock. Connected members may cover small shortfalls, but destination borrowers remain below the 0.30 attribution threshold. Tests graph overreaction.	Graph overreaction / false positive control
individual_deterioration	Gradual borrower-specific decline over ~6+ months with no meaningful network contribution. Tests independent deterioration.	Independent deterioration vs. propagation
propagating_stress	Source loses cash buffer; shortfalls create group/connected financial burden through financial mechanics; destination later deteriorates with lag. Hidden lineage records the pathway.	Early-warning detection of a genuine simulated transmission pathway

Generator/detector separation: propagation must emerge from financial mechanics such as shortfall coverage, cash-buffer depletion, shared income dependence, affordability deterioration, and delayed liability effects - never from the detector formula.

3.3 Hidden ground-truth lineage

- Every simulated deterioration retains hidden lineage: source event, mechanism, affected entity, lag, and attributable contribution share. This stream is evaluation-only and is never exposed as a model feature.
- For nonlinear attribution, compute the network contribution using a same-seed/state counterfactual rerun with network-transfer mechanisms disabled. The network contribution is the downstream shortfall difference between the full world and the network-disabled counterfactual, normalized into a contribution share.
- This gives the evaluation harness an independent ground truth for propagation attribution while keeping detector features restricted to what would have been knowable at prediction time.

4. Analytics, Stress Definitions & Targets

4.1 Operational stress definition

REQUIRED DECISION / Define stress from observed outcomes at time t , not from model outputs. Use a transparent rule combining repayment behavior and/or sustained cash-flow strain with persistence / repeated evidence. The exact thresholds are configurable in config/targets.yaml and frozen before held-out evaluation.

Target	Label construction	Feature time window
Current Stress	Observed stress state at assessment time t under frozen repayment / cash-flow criteria.	Strictly before t for all prediction features; same-time evidence defining the label is excluded.
Next-Period Stress	Currently non-stressed at t , becomes stressed at $t+1$ or configured horizon.	At or before t .
Propagation Vulnerability	Currently non-stressed at t , later becomes stressed, and hidden lineage shows a network contribution share ≥ 0.30 .	At or before t only; interval after t and before outcome is completely excluded; lineage/outcome are evaluation-only.
Healthy-Borrower Protection	Healthy at t , flagged/recommended for intervention, and not stressed within the evaluation horizon; analyze by exposure level.	Only prediction-time-valid inputs.

4.2 Model ladder

Model	Inputs	Research question
M0	Simple delinquency heuristic / recent repayment delay	How well can an obvious baseline perform?
A - Individual	Income, expenses, cash buffer, repayment trajectory, debt burden, seasonality	Does individual financial history improve early detection?
B - Graph Enhanced	A + pre- t relationship / group / shared-source features	Does network context add signal beyond individual history?
C - Propagation Aware	B + propagation exposure features built only from valid pre- t information and OOF / earlier-period neighbor estimates.	Does explicit propagation modeling improve early identification of network-vulnerable borrowers?

No scenario labels as features: scenario_family, shock_type, hidden lineage, future state, future stress, post- t observations, and simulator-only variables are prohibited predictive inputs.

4.3 Final feature timing contract

- Current Stress: label uses state at t ; prediction features are strictly pre- t .
- Next-Period Stress: label is future relative to t ; features may use information available at or before t .
- Propagation Vulnerability: predictor sees only information available at or before t ; no data from (t , outcome horizon] may enter prediction features.
- Healthy-Borrower Protection: measured from prediction-time features and future outcomes only in the evaluation stream.

ZERO LOOK-AHEAD

At timestamp t , every feature must be computable from observations available at or before t according to the feature contract. Features may use prior model predictions generated from prior information; they may not use future ground-truth outcomes.

5. Leakage, Circularity & Causality Governance

Rule	Implementation requirement	Automated test
Future-label ban	No $t+1$ stress, future repayment, future income/expenses, future relationship status, or scenario label in features	Feature provenance audit + forbidden-column test
Model-label separation	Propagation target comes from future observed outcome + hidden lineage / counterfactual attribution, never model scores.	Target construction unit test
Generator/detector separation	Generator and simulator share mechanics.py; data.py owns world generation + hidden lineage. Only targets.py / evaluate.py may import hidden lineage.	Import/dependency test
Chronological split	Train < validation < held-out test with purge gap $\geq$ max propagation lag + prediction horizon; episodes cannot cross boundaries.	Temporal split test
Prediction-time freeze	Stress threshold, 0.30 primary lineage threshold, transmission factors, calibration, and alpha fixed before held-out test.	Config freeze test
No circular scoring	Risk score cannot be fed back into its own feature computation at the same timestamp	Dependency graph test
Observed vs estimated	Every displayed quantity is tagged as observed, estimated, exposed, simulated, or future observed outcome	Schema/UI contract test
OOF / historical neighbor estimates	Model C neighbor stress estimates are OOF or from earlier-period fits only.	Prediction provenance test
Attribution threshold	Primary target threshold is preregistered at 0.30; 0.20 / 0.50 are sensitivity checks only.	Config freeze + robustness test

6. Propagation Engine

6.1 Group-first exposure model

$\text{group_liability_exposure}(g,t) = \text{bounded_function}(\text{pre-}t \text{ covered_shortfall}(g,t) / \text{group_capacity}(g,t))$

$\text{channel_exposure}(B,t,c) = \text{valid_source_estimate}(t) \times \text{relationship_weight}(B,t,c) \times \text{transmission_factor}(c)$

$\text{network_vulnerability}(B,t) = 1 - \text{Product over distinct semantic channels } c \text{ of } (1 - \text{clipped_channel_exposure}(c,B,t))$

The JLG/group liability mechanism is compressed once per group and passed once to each member. Do not aggregate dozens of equivalent member-level liability edges.

6.2 Model C dependency and calibration contract

Exposure computation must exist before Model C because its outputs are Model C features. Path extraction is downstream visualization, not a prerequisite for training.

Neighbor stress estimates must be out-of-fold within training periods or generated from models fit only on earlier periods. In-sample predictions from a model trained on the same target rows are prohibited.

Calibration is validation-only. Individual stress and Model C outputs are calibrated against their own target events. Network exposure is calibrated against Propagation Vulnerability only when presented as probability-like; unlike-event quantities must not be blended as though they were equivalent probabilities.

Model C calibrated probability is the single operational risk output. It drives risk badges, state transitions, and recommendations.

$\text{decomposition_risk}(B,t) = \alpha \times \text{calibrated_individual_stress} + (1-\alpha) \times \text{calibrated_network_exposure}$

The alpha decomposition is explanation-only and cannot independently drive actions. Freeze calibration parameters, alpha, transmission factors, and action thresholds before held-out testing.

Caps are allowed only as explicit, documented safety bounds. Group-level risk must avoid double counting the same members through multiple graph paths.

6.3 Propagation path representation

source_borrower / source_group: origin of modeled exposure; destination_borrower: potentially exposed member; channel: group_liability / shared_income / guarantee / centre_context; lag_estimate: estimated time between source signal and destination exposure; exposure_strength: pre-t modeled exposure, not realized future impact; destination_state: healthy / watch / vulnerable / stressed; path_confidence: model evidence strength, not causal certainty.

6.4 State machine

Healthy -> Watch -> Vulnerable -> Stressed

Transitions must use observed and estimated quantities with persistence / hysteresis so one noisy observation does not cause repeated state flipping. State labels do not imply causality.

Field	Example meaning
source_borrower	A
destination_borrower	B
channel	group_liability / shared_income / guarantee

Field	Example meaning
lag_estimate	2 weeks
exposure_strength	0.63
source_state	estimated stressed
destination_state	healthy / watch / vulnerable / stressed
path_confidence	model estimate, not causal certainty

7. Ripple Simulator & Intervention Studio

7.1 Ripple Simulator

- Simulator mechanics: Apply the selected shock to a cloned baseline state and advance the clone with the same forward cash-flow / liability mechanics used by the synthetic world. The simulator may import mechanics.py, but it must never read hidden lineage or call evaluation-only attribution logic.
- The simulator is a deterministic counterfactual mechanism, not a causal forecast. Cap request scope / horizon so a serverless request remains bounded.

Step	Requirement
1. Select	Choose borrower, group, or economic source.
2. Shock	Choose income drop, expense surge, repayment delay, income-source loss, or regional downturn.
3. Parameters	Magnitude, duration, affected channel, start period, and random seed.
4. Clone	Clone baseline state; never mutate source data.
5. Recompute	Recalculate financial state, model estimates, propagation exposure, and group impact.
6. Visualize	Animate affected nodes/edges; distinguish direct exposure from second-order effects.
7. Compare	Before/after metrics, affected borrowers, healthy borrowers affected, peak exposure, time-to-impact.
8. Reset	Return to baseline deterministically.

Determinism: Same baseline + shock + parameters + seed must produce identical results. Different seeds can support robustness analysis. Baseline state must never be mutated.

7.2 Intervention Studio

Intervention	Purpose	Assumption examples
No Action	Baseline for comparison	No change
Monitor	Increase review frequency without	Detection benefit only

Intervention	Purpose	Assumption examples
	changing repayment terms	
Targeted Borrower Support	Assist only directly affected borrower	Recovery probability, delay reduction
Temporary Repayment Flexibility	Short relief window	Burden reduction, duration, recovery effect
Restructuring	Alter schedule for a selected borrower/group	New payment profile, recovery assumptions
Group-Level Intervention	Act on shared-risk mechanism	Coverage / coordination assumptions

ASSUMPTION TRANSPARENCY

Intervention effects are modeled assumptions, not learned causal effects. The UI must show the assumptions table, confidence/uncertainty, and sensitivity under conservative/base/optimistic parameter sets.

8. Product Experience & UX

SIX PRODUCT AREAS

The product has six areas. They are not six independent apps; all views consume the same temporal state, graph, propagation, simulation, and intervention services.

Area	Primary interaction	Judge-facing moment
1. Command Center	Portfolio/group health, alerts, hotspots, emerging stress	Instantly understand what is happening and where
2. Borrower Intelligence	Timeline, cash flow, repayment, stress decomposition, evidence	Explain why this borrower is stressed / vulnerable
3. Network Intelligence	2D default graph; optional 3D showcase mode; filters and path tracing	Show how exposure can move through a group
4. Ripple Simulator	Interactive shock + animated propagation + scenario comparison	Create a memorable before/after demo
5. Intervention Studio	Compare interventions, assumptions, sensitivity, healthy-borrower impact	Demonstrate targeted action rather than blanket intervention
6. Model & Impact Lab	M0/A/B/C, scenario metrics, confidence intervals, fairness, robustness	Prove that the intelligence layer adds measurable value

8.1 Operational UX defaults

- 2D graph is the default operational network view because it prioritizes readability and device performance.
- 3D network is a showcase mode for presentations, with a toggle and the same underlying graph state.

- Every risk badge expands into an evidence story: observed facts → estimated exposure → modeled vulnerability → recommended action.
- No Action must be a valid explicit recommendation for healthy borrowers.

8.2 Visual language

Brand: NEXUS / Financial Resilience Intelligence

Aesthetic: Dark analytical workspace, restrained glass surfaces, high information density, smooth but purposeful transitions, accessible contrast, clear state semantics.

Avoid: Dashboard clutter, decorative 3D everywhere, opaque single-number risk scores, animation that obscures evidence, UI elements that look functional but have no underlying logic.

9. Explainability, Fairness, Inclusivity & Responsible Data

UI wording must never imply that an exposure estimate is a realized event or that an association constitutes a causal finding. Use the epistemic label and evidence type consistently in every view.

9.1 Explainability contract

Label	Meaning
OBSERVED	Directly recorded historical fact, e.g. recent repayment delay or income decline.
ESTIMATED	NEXUS model output based on pre-t information.
EXPOSED	Modeled network relationship/exposure, not a realized future event.
SIMULATED	Counterfactual result under a user-selected scenario.
FUTURE OBSERVED	Outcome later observed in the evaluation stream; never available at prediction time.

9.2 Healthy-borrower protection

- Healthy + connected must not automatically become intervention-worthy.
- Healthy + weak exposure, healthy + strong exposure, stressed + isolated, seasonal stressed + connected, and genuinely propagated stress must all appear in fairness test cases.
- Report unnecessary flagging and intervention rates among currently healthy borrowers.

9.3 Inclusion / low-bandwidth

- Provide a lightweight mode that minimizes heavy graph animation and image assets.
- Use plain-language explanations and architect the explanation layer so translations into major Indian languages can be added without rewriting the analytics core.
- Show borrower-facing explanations as a product capability, not only internal risk labels.

9.4 Responsible data handling

The prototype should follow data-minimization, purpose-limitation, role-based access, audit logging, and synthetic/demo-data defaults. For Vercel, persistent audit logs / role state must use an external persistent store

or be explicitly stubbed for the prototype; never rely on mutable local filesystem state. The architecture should be compatible with responsible processing of digital personal data under India's data-protection framework. This is product design guidance, not legal advice.

10. Evaluation & Statistical Design

Evaluation family	Metrics / output
Current Stress	Precision, recall, F1, PR-AUC where appropriate
Next-Period Stress	PR-AUC, recall at operational precision, calibration / reliability
Propagation Vulnerability	PR-AUC, early-warning lead time, path detection precision
Scenario robustness	Separate performance on seasonal, isolated, individual-deterioration, propagating-stress
Healthy-borrower protection	False flags / intervention rate among currently healthy borrowers
Group impact	Group-level stress concentration, number of newly vulnerable members, exposure distribution
Robustness	10+ seeds, mean/median, spread and confidence intervals
Ablation	M0 vs A vs B vs C on the same chronological held-out protocol

STATISTICAL POWER

Do not over-interpret a tiny held-out propagation subset. Increase the number of episodes / borrowers as needed, report event counts and confidence intervals, and expose the count next to each metric.

10.1 Scenario-balanced test design

1. Create scenario episodes inside each chronological split without allowing an episode to straddle train/validation/test boundaries, and insert a purge gap of at least maximum propagation lag + prediction horizon between splits.
2. Keep scenario distribution visible in the evaluation export.
3. Aggregate metrics across seeds only after per-seed metrics are computed.
4. Use at least 10 world-generation seeds; compute per-seed metrics before aggregate summaries; use confidence / bootstrap intervals and display sample counts.

11. System Architecture & API

11.1 Module boundaries

Module	Responsibility
src/mechanics.py	Shared forward cash-flow and liability mechanics used by generator and simulator; contains no hidden lineage or detector scoring.

Module	Responsibility
src/data.py	Synthetic microfinance world generation, repeat loans, scenario generation, and hidden lineage creation. Hidden lineage is evaluation-only.
src/features.py	Strictly time-valid temporal / financial features with provenance.
src/graph.py	Branch -> Centre -> JLG hierarchy, relationship semantics, and group exposure inputs.
src/targets.py	Stress targets, propagation attribution from future outcomes + lineage / counterfactuals, and frozen config validation.
src/propagation.py	Group-first exposure computation required before Model C; distinct-channel aggregation; downstream path extraction helpers.
src/calibration.py	Validation-only calibration for model outputs and probability-like exposure displays.
src/simulator.py	Clone state, apply shock, advance with mechanics.py, recompute NEXUS estimates deterministically; never read hidden lineage.
src/interventions.py	Counterfactual intervention assumptions, sensitivity, and comparison.
src/train.py	M0/A/B/C training and OOF / earlier-period neighbor estimate generation; frozen model artifacts.
src/evaluate.py	Chronological evaluation, purge gaps, 10+ world seeds, confidence intervals, scenario/fairness analysis, null-result reporting; may read lineage.
src/explain.py	Evidence chains and epistemic labels.
src/api.py	FastAPI endpoints, validation, typed responses, stateless serving contracts.
src/static/*	Six product areas, 2D graph, optional 3D, simulator, intervention, evaluation dashboards.
tests/*	Timing, generator isolation, lineage access, stacking provenance, aggregation, calibration, determinism, fairness, API, performance, deployment smoke tests.

11.2 API contract

Endpoint	Purpose
GET /health	Service health / version
GET /portfolio/summary	Portfolio and group health summary
GET /borrower/{id}	Borrower intelligence and evidence timeline
GET /group/{id}	Group health, members, exposure, propagation paths

Endpoint	Purpose
GET /network	Filtered network data for 2D/3D views
POST /simulate	Run deterministic shock scenario on cloned state
POST /intervene	Evaluate intervention scenarios and sensitivity
GET /evaluation	M0/A/B/C metrics and scenario breakdown
GET /assumptions	Current frozen model/intervention assumptions

12. Testing & Definition of Done

12.1 Mandatory tests

- Temporal integrity: every feature at t uses only information permitted by the target timing contract.
- Current-Stress adversarial timing: same-time evidence used to define the label cannot enter Current Stress features.
- Generator / simulator boundary: data.py may import mechanics.py but not detector scoring; simulator.py may import mechanics.py but not hidden-lineage or evaluation-only attribution logic.
- Lineage-access restriction: only targets.py and evaluate.py may access hidden lineage; features, graph, propagation, train, API, and simulator must not.
- Target independence: Propagation Vulnerability labels come from future observed outcomes plus hidden lineage / counterfactual attribution, never model scores.
- OOF / historical stacking: Model C neighbor estimates are out-of-fold or earlier-period predictions only, with provenance recorded.
- No circular scoring: same-timestamp risk outputs cannot become inputs to their own feature computation.
- Group aggregation: JLG liability is counted once per semantic channel; large groups do not saturate vulnerability solely through member count.
- Calibration: Model C calibration and any probability-like network exposure calibration are validation-only and frozen before held-out evaluation.
- Attribution threshold freeze: primary meaningful_contribution_share remains 0.30; 0.20 and 0.50 are sensitivity checks only.
- Purge gaps: every train/validation/test boundary has a gap of at least maximum propagation lag plus prediction horizon.
- Simulation determinism: identical input/seed gives identical output; baseline state is never mutated; hidden lineage is never read.
- Fairness cases: healthy connected / low exposure / high exposure / stressed isolated / seasonal correlated / genuinely propagated cases.
- API schema and frontend state consistency tests.
- Performance smoke test for default dataset, 2D network rendering, and bounded simulator/intervention requests.
- Vercel deployment smoke test: production build succeeds, core endpoints respond, and no request depends on persistent local mutation.

12.2 Definition of Done

Area	Done means
Data	300-600 borrower-capable realistic synthetic world with Branch/Centre/JLG hierarchy, multiple loan cycles, external debt, four scenario families, and hidden lineage.
Analytics	M0/A/B/C implemented; targets independently defined; timing / stacking / leakage tests pass.
Propagation	Group-first exposure before Model C, distinct-channel aggregation, calibration, 0.30 preregistered attribution threshold, path extraction later.
Simulation	Deterministic clone-and-shock engine using mechanics.py; baseline never mutated; lineage never read.
Intervention	At least 4 intervention scenarios + explicit assumptions + conservative/base/optimistic sensitivity.
UX	Six product areas functional; 2D default graph; 3D showcase only after core stability.
Evaluation	10+ world-generation seeds, purge gaps, confidence intervals, scenario/fairness breakdown, event counts, null-result reporting.
Responsible design	Healthy-borrower protection, low-bandwidth mode, explanation labels, privacy-aware defaults, external persistence or explicit prototype stubs.
Deployment	Local production build + Vercel smoke test; serving bundle excludes training-only dependencies where practical; bounded simulation requests; no persistent local state.
Docs	README, architecture, setup, data dictionary, evaluation methodology, limitations, deployment runbook, and demo path.

13. M# Traceability, SDG & Monetization

M# judging dimension	NEXUS evidence
Problem understanding	Explicit distinction between isolated, independent, and network-vulnerable stress
Innovation	Propagation-aware financial resilience layer + causal-separation simulation + healthy-borrower protection
Implementation quality	Modular engine, deterministic simulator, temporal integrity tests, M0/A/B/C ablation
Real-world applicability	Group liability, repeat loans, affordability, shared income,

M# judging dimension	NEXUS evidence
	regional shocks
Inclusivity	Avoids blanket intervention; low-bandwidth and borrower-facing explanation pathways
Monetization	B2B SaaS/API intelligence layer for MFIs and NBFC-MFIs
Marketing / demo	Command Center → Borrower → Network → Ripple → Intervention → Impact narrative
Prototype	Interactive working web app with real underlying logic

13.1 Monetization model

BUSINESS POSITIONING

NEXUS can be positioned as a B2B resilience-intelligence layer that integrates with an MFI/NBFC-MFI loan-management stack. Potential commercial models include pricing per active borrower, portfolio-size tiers, or enterprise/API licensing. The prototype need only demonstrate the buyer, value unit, integration point, and expansion path.

Core buyer value: Reduce avoidable portfolio-level stress by identifying emerging network exposure early and targeting limited intervention to the borrowers/mechanisms most likely to matter. Value is framed through operational units such as PAR-30 concentration, newly vulnerable borrowers, intervention volume, and healthy-borrower false intervention rate.

13.2 Real-world grounding references

- RBI, Regulatory Framework for Microfinance Loans (2022) and updated FAQ (Jan 30, 2025): household-level income/indebtedness assessment and the 50% cap on monthly household loan repayment obligations for low-income households are relevant product-grounding considerations. Source: RBI FAQ, <https://www.rbi.org.in/commonperson/English/Scripts/FAQs.aspx?Id=3366>
- RBI, Annual Report 2021-22: the 2022 microfinance framework emphasized customer protection and harmonized regulation across regulated entities. Source: RBI, <https://www.rbi.org.in/Scripts/AnnualReportPublications.aspx?Id=1343>
- Government of India, Digital Personal Data Protection Act, 2023: establishes the legal framework for processing digital personal data; NEXUS should treat privacy, purpose limitation and access controls as design requirements. Source: Ministry of Law & Justice / MeitY, <https://www.meity.gov.in/writereaddata/files/Digital%20Personal%20Data%20Protection%20Act%202023.pdf>

14. Agent Handoff & Implementation Contract

SOURCE OF TRUTH / This PRD is the canonical specification. Agents may improve implementation details, performance, and internal APIs, but may not silently redefine target semantics, timing rules, lineage attribution, model-score semantics, simulation mechanics, evaluation protocol, deployment constraints, or

the scope cut line.

14.1 Implementation order

1. Inspect copied RippleGuard repository, Git remotes, and current architecture.
2. Create/validate configs and schemas for data, targets, model, propagation, calibration, intervention, evaluation, and deployment.
3. Implement src/mechanics.py as the shared forward financial-mechanics layer.
4. Implement realistic synthetic world generation in src/data.py using mechanics.py; create hidden lineage separately; enforce generator/detector separation.
5. Construct targets immediately after generation; fix primary lineage threshold at 0.30; implement counterfactual attribution.
6. Implement target / lineage validation tests.
7. Implement strictly time-valid feature engine + provenance + adversarial same-time tests.
8. Implement Branch/Centre/JLG graph and group-first exposure computation.
9. Establish chronological train/validation/test splits with purge gaps.
10. Implement M0 and Model A.
11. Generate OOF / earlier-period neighbor stress estimates with explicit fit-window provenance.
12. Implement Model B and Model C.
13. Implement validation-only calibration; freeze all test-time parameters.
14. Run primary evaluation across 10+ world-generation seeds with CIs, scenario breakdown, fairness, event counts, ablation, and null-result reporting.
15. Implement downstream propagation path extraction / visualization.
16. Implement deterministic Ripple Simulator using mechanics.py on cloned state; no lineage access.
17. Implement Intervention Studio + sensitivity.
18. Implement explainability, epistemic labels, healthy-borrower safeguards, and borrower-facing explanation layer.
19. Implement FastAPI and stateless serving contracts.
20. Implement six-area UI with 2D default graph.
21. Add 3D showcase only after core functionality is stable.
22. Run deployment-oriented performance checks before packaging.
23. Add / verify Vercel deployment configuration and clean production build.
24. Run full test suite + production smoke test.
25. Freeze metrics / assumptions; document limitations and demo path.
26. Polish README, architecture diagram, screenshots, and M# pitch materials.

14.2 Agent responsibilities

Agent	Primary role
Claude Code / Opus 4.6 Thinking	Architecture review, codebase forensics, risk identification, analytical correctness, test design, final review
Antigravity / Gemini 3.1 Pro High	Implementation, frontend/UX, integration, performance, test execution, docs, polish

14.3 Vercel deployment contract

- Build NEXUS so the final web application can deploy to Vercel without an architectural rewrite: frontend + FastAPI backend using Vercel-supported project configuration.
- Training and full evaluation are offline / build-time workflows. Production endpoints load frozen model / calibration artifacts and run lightweight inference, propagation exposure calculations, bounded simulations, and intervention calculations.
- API requests are stateless. Do not rely on mutable local filesystem state across invocations. Persistent audit logs and role state must use an external persistent store, or be explicitly stubbed in the prototype.
- Keep training-only dependencies out of the serving bundle where practical. Measure the deployed Python bundle and function duration early; do not wait until final deployment to discover size or execution-limit problems.
- Bound /simulate and /intervene request horizon, node count, and workload so forward mechanics + inference fit comfortably within the serverless execution budget. Load packaged frozen baseline artifacts rather than regenerating the full world per request.
- Vercel acceptance test: production build succeeds; frontend loads; /health succeeds; all core endpoints return typed responses; repeated identical simulations are deterministic; no endpoint depends on persistent local mutation.

15. Skeptical-Judge Review Checklist

1. Can the team explain exactly how stress is defined and why Current Stress features are strictly pre-t?
2. Can the team explain why Next-Period Stress may use data at or before t?
3. Can the team prove Propagation Vulnerability features use only prediction-time information?
4. Can the team prove propagation labels come from hidden lineage / counterfactual attribution rather than model output?
5. Can the team prove generator, mechanics, simulator, and detector boundaries prevent circularity?
6. Can the team explain why the primary attribution threshold is fixed at 0.30 and how 0.20 / 0.50 sensitivity is reported?
7. Can the team distinguish correlation from network transmission, especially during seasonal downturns?
8. Can the team explain why JLG group liability is aggregated once rather than counted as many equivalent member edges?
9. Can the team prove Model C neighbor estimates are OOF or earlier-period and not in-sample?
10. Are calibration parameters and action thresholds validation-only and frozen before held-out testing?
11. Is calibrated Model C the single operational score, with alpha used only as an explanatory decomposition?

12. Can a healthy connected borrower remain at No Action, including under high modeled exposure?
13. Are intervention effects presented as assumptions with sensitivity rather than learned causal facts?
14. Are M0/A/B/C comparisons evaluated under identical chronology with purge gaps, event counts, and confidence intervals?
15. Are 10+ world-generation seeds used rather than relying only on model-randomness seeds?
16. Does the simulator use shared forward financial mechanics, remain deterministic, avoid hidden lineage, and avoid mutating baseline state?
17. Does the synthetic world resemble an actual microfinance operating structure with Branch/Centre/JLG hierarchy, group liability, repeat loans, and affordability constraints?
18. What is the buyer, what is the value unit, and how does NEXUS integrate with an existing LMS / workflow?
19. Can the operational interface run in low-bandwidth mode and preserve clear epistemic labels?
20. What is the explicit cut line if implementation time becomes constrained?

16. Final Scope Cut Line

CORE - must be working

- - Command Center
- - Borrower Intelligence
- - Network Intelligence with 2D graph
- - Ripple Simulator
- - Intervention Studio
- - Model & Impact Lab
- - Temporal / leakage / stacking / determinism / fairness tests
- - M0/A/B/C evaluation with honest held-out metrics
- - Vercel-ready stateless serving path

STRETCH - only after core is stable

- - 3D network showcase mode
- - Borrower-facing portal polish
- - Expanded vernacular localization
- - Additional advanced scenario presets / animations

CUT RULE

If time becomes constrained, remove stretch features before compromising analytical correctness, target independence, leakage tests, evaluation, deployment reliability, or the six core product areas.

FINALIZATION STATUS

This is the clean frozen implementation specification. Further changes should be limited to genuine contradictions discovered during the final audit or implementation, not new feature expansion.